```
      currentCount: Map[String, Int],
      newCount2Add: Map[String, Int]): Map[String, Int] = ...
}
```

❶ State that contains the counted words

❷ State that contains the lag and total aggregated words

❸ After a successful request, aggregation

❹ After a failed request, unprocessed text goes back to lag.

❺ Merge function to add the new count to current

Let's take another look at the master business logic; see figure 8.13. The worker has only one message type in its protocol, with the text to process and the reference to reply to the master:

```
final case class Process(text: String, replyTo: ActorRef[Master.Event])
      extends Command with CborSerializable
```

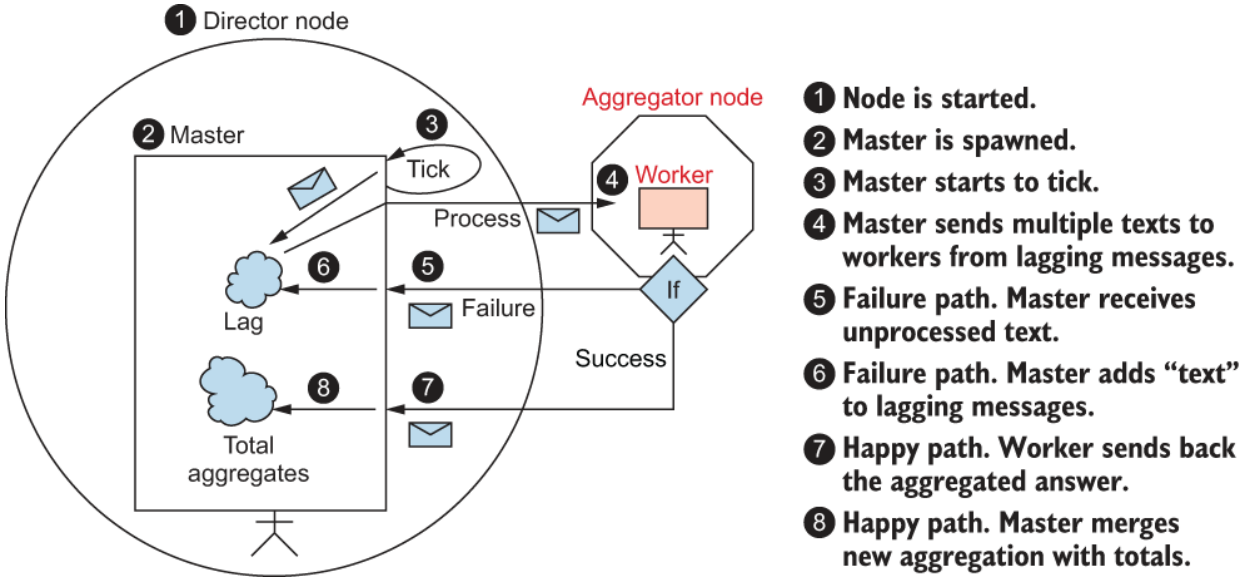


Figure 8.13 Master business flow

During setup, the worker registers itself in the registration key so the master's router can find it later. After setup, each time it receives a